

CSC-491-Final

Matt Liou, Aaditya Mishra, Nick Bailey

December 2025

Contents

1	Introduction	4
2	System Design: Basic Loop	4
2.1	Evolution Log Management	5
2.2	Leveraging Logs for Optimization	5
3	Sensitivity of Performance to Reasoning Strategy (Direct vs CoT vs ToT)	6
3.1	Overall Quality	6
3.2	Tail Behavior and Frontier Policies	6
3.3	Robustness and Failure Modes	6
3.4	Exploration vs. Stability	6
3.5	Why We Favor CoT	6
3.6	Visualization	7
3.6.1	Mean IPC with Error Bars	7
3.6.2	Best-So-Far IPC vs. Iteration	7
4	Sensitivity to Fallback Strategy	7
4.1	Iteration-Level Results	7
4.2	Aggregate Summary	8
4.3	Visualization	8
4.4	Interpretation	8
5	Sensitivity to Mutation	8
	Rate Schedules	8
6	Sensitivity of Performance to Prompting Style (Concise vs Complex)	10
6.1	Per Iteration IPC	10
6.2	Summary Statistics	10
6.3	Interpretation	10
6.4	Takeaways	11
7	Reinforcement Learning for Prompt Selection	11
7.1	Bandit Integration into the Search Loop	11
7.2	Reward Signal and Learning	11
7.3	Comparison to the Fixed CoT Baseline	11
7.4	Takeaways	11
7.5	Visualization	11
8	Structured Policy Explainer	12
9	Surrogate IPC Model and Accuracy	12
9.1	Training Data and Feature Construction	12
9.2	Model Architecture and Training	13
9.3	Offline Evaluation Procedure	13
9.4	Role in the Overall Search Loop	14
10	LLM-Discovered Hybrid Replacement Policy on Full SPEC	14
10.1	Novelty and Motivation	14
10.2	Discovery Process	14
10.3	Final Performance on Full SPEC CPU 2006	14
10.4	Qualitative Analysis of the CSF Policy	15
10.5	Summary	15

11 Search Dynamics and Policy Distribution	15
11.1 Iteration-Level Search Dynamics	16
11.2 Impact of Prompt Design and the Over-Prompting Reset	17
11.3 Frontier Structure and Policy Distribution	17
11.4 Interpretation: Why IPC Improves as Hit Rate Falls	18
12 Area Table Breakdown	18
13 Conclusion	18
14 References	19

1 Introduction

Optimization of the performance of modern computers has been a goal of scientists and engineers since the day the first modern computer was released. With each release of new chip architecture, the capabilities of a computer system increases. And this demand for optimized computer performance has only increased with the rise and popularity and usability of Artificial Intelligence. Companies like NVIDIA, Intel, and AMD have been in a race to create new chip designs which optimize the speed and performing power of their computer chips.

However, hardware improvements are only one part of the equation for creating more powerful computing systems. Another part of the equation lies in the memory hierarchy. Caches, positioned between the processor and main memory, play an integral role in reducing access latency and improving overall throughput. And while increasing the cache size seems like the obvious item to improve, software cache replacement policies play an even bigger role in making sure that caches are being used effectively.

In Homework 1, we used an early form of Cacheforge, which took the code created by the LLM and simulated it on 5 traces in ChampSim. However, there were a couple of concerns that were brought up with the original Cacheforge model. Firstly, the cacheforge model lacked persistent memory. It didn't track what changes were being made, and how these changes affected the hit rate and IPC. This led to inconsistent improvements and often the same mistakes being made over and over again. The second issue was that the Cacheforge system was unguided. It started with no starting replacement policy and was instructed to keep on iterating replacement policies without any real, clear guidelines. The instructions given were very vague and led the system to make a lot of assumptions and mistakes. The final was that every policy created had to be simulated on the full spec to get an average IPC value. While this was okay when ChampSim only had 5 different branches to test on, for the Final Project, we are expected to simulate on the full SPEC CPU 2006 benchmark, which can take an hour or two to run.

The objective of this project was to take the shortcomings of Homework 1, fix them, add different elements that we learnt in class, and ultimately get a replacement policy that beats the baseline policy given. For this project, we decided to implement a couple of things: surrogate models, chain of thought, and graph of thought. With these changes, we hoped to beat the baseline and ultimately find a more optimal solution to replacement policies.

2 System Design: Basic Loop

CacheForge wraps the LLM in a closed-loop “experiment engine” that repeatedly proposes new cache replacement

policies, runs them in ChampSim, and updates a persistent memory of what was tried and how well it worked. At a high level, each outer iteration of the loop proceeds as:

1. **Context assembly.** Retrieve relevant prior policies, prompts, and results from the evolution logs (Section 2.1) using a lightweight RAG stack (Hybrid-Memory + ExperimentRAG).
2. **Prompt/mode selection.** Choose between exploration and exploitation based on the current best score and a small exploitation budget, and build the corresponding prompt.
3. **Reasoning and code generation.** Run the LLM to produce one or more candidate policies, each with a name, description, C++ implementation, and structured explainer.
4. **Surrogate-guided triage.** Optionally score candidates with the surrogate model and select a small subset for full simulation.
5. **Compilation and simulation.** Apply the patch to the code base, compile ChampSim, and run the chosen policy on all simpoints with traces.
6. **Logging and update.** Aggregate per-workload metrics, update the experiment database and lineage logs, refresh the surrogate training set, and update the best-so-far score and explore/exploit mode.

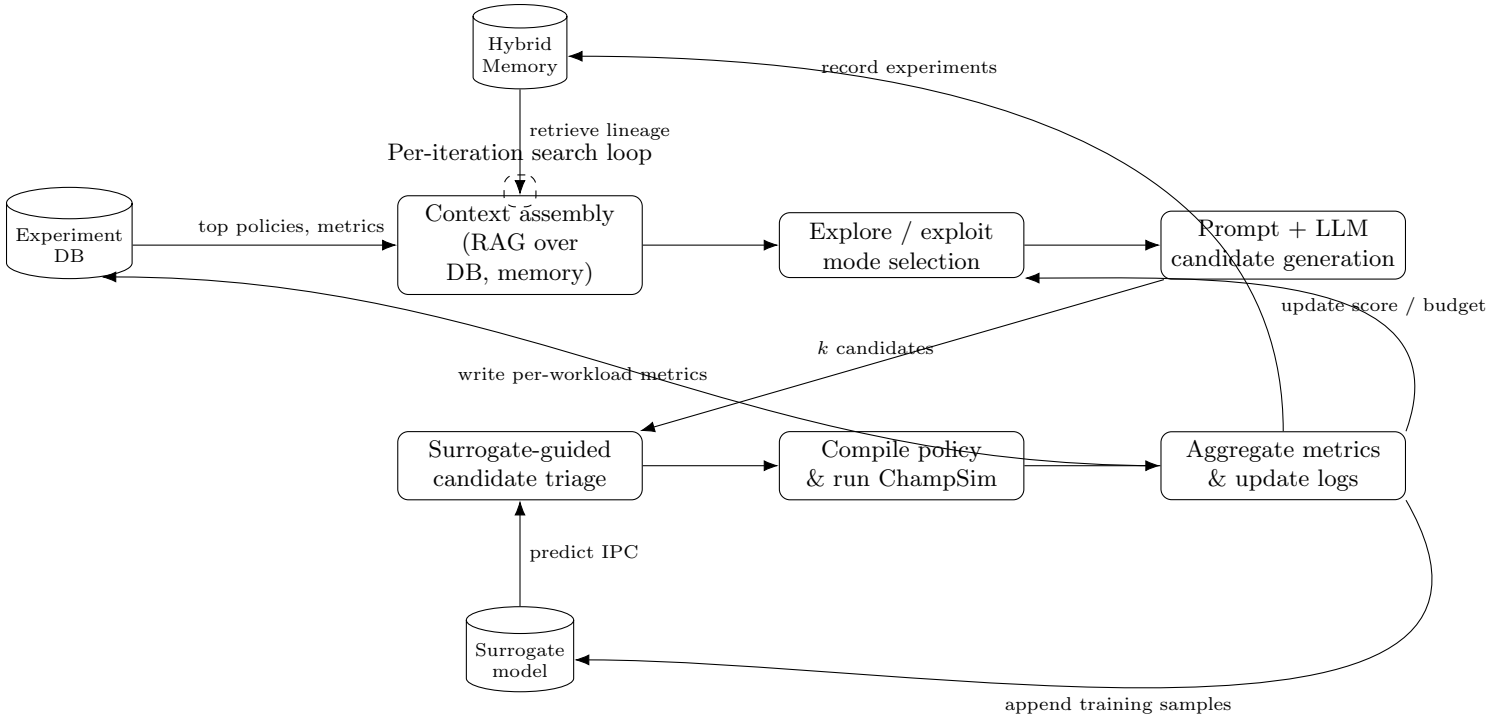


Figure 1: CacheForge system design based on `run_loop.py`. Each iteration assembles context from prior experiments, generates candidate policies with the LLM, uses a surrogate model to triage candidates, runs ChampSim on the chosen policy, and logs metrics to drive the next iteration.

2.1 Evolution Log Management

CacheForge writes two complementary artifacts per run of `run_loop.py`:

Experiment log (JSON/DB). Each evaluated (or attempted) policy is recorded with a minimal, simulation-facing schema: `id`, `iteration`, `policy_name`, `policy_desc`, `compile_ok`, `avg_ipc`, and a `per_workload` dictionary (e.g., `{ "omnetpp": { "hit_rate": ..., "ipc": ... }}`). When only some benchmarks provide traces, only those workloads appear and the all aggregate reduces to the available set.

Lineage graph (JSON + DOT). Each iteration also updates a lineage store via `LineageTracker`: nodes carry `id`, `name`, `iteration`, `model`, `code_path`, `code_hash`, `metrics.avg_ipc` (or null if not simulated), `per_workload status`, lightweight `storage` (Overhead Break-down for `Compressedtestestimate_kb`, `note`), `parent_id`, and `timestamp`. Candidates that the surrogate ranks out before simulation are marked [SKIP] and logged with `avg_ipc = null` and `per_workload: ["status": "skipped: ranked_out"]`. Edges optionally include a compact diff summary from `summarize_code_mutation` when a parent compiled previously.

Together these logs let us (i) reconstruct best-so-far tra-

jectories without re-running ChampSim, (ii) query per-workload behavior, and (iii) visualize the search tree with compile/skip outcomes.

2.2 Leveraging Logs for Optimization

The same artifacts drive control in `run_loop.py`:

- **Surrogate.** The surrogate reads the experiment log’s structured features (explainer fields, storage estimate, basic metrics) to score fresh candidates; low-ranked ones are skipped and never simulated.
- **Mode/budget updates.** After each compiled run, the new `avg_ipc` updates the best-so-far and the exploitation budget used by the next iteration’s explore/exploit switch.
- **Context retrieval.** The DB + simple JSON feed a lightweight RAG layer so prompts can include recent, relevant successes/failures instead of starting cold each time.

This keeps simulation time focused on promising regions (via skips), preserves a queryable audit trail (experiment log), and provides a compact search narrative (lineage) that the loop can use to iterate more efficiently.

3 Sensitivity of Performance to Reasoning Strategy (Direct vs CoT vs ToT)

We compared three prompting strategies inside the CacheForge search loop on a reduced set of workloads for faster turnaround. For the sensitivity experiments in this section, we enabled the `few.workloads` configuration and restricted evaluation to a single representative SPEC CPU 2006 workload (`omnetpp`), using the same ChampSim settings and outer loop across all prompting modes. Specifically, we evaluated:

1. **Direct prompting** (DIRECT): the model is asked to “write a better policy” with no explicit reasoning trace.
2. **Chain-of-Thought** (CoT): the model first produces a linear explanation of its reasoning, then emits a C++ patch.
3. **Tree-of-Thought** (ToT): the model explores multiple candidate branches, prunes them, and finally emits a patch.

3.1 Overall Quality

For each method, we aggregated the average IPC of all valid policies. Table 1 summarizes the basic statistics.

Table 1: Summary statistics over valid policies for each reasoning strategy (omnetpp-only, `few.workloads` config; not full SPEC CPU 2006 suite).

Method	n	Mean	Median	Min	Max	Std.	Q_{10}	Q_{90}
DIRECT	47	0.455524	0.454878	0.432251	0.471578	0.010397	0.440301	0.470639
CoT	100	0.460180	0.460266	0.430182	0.472265	0.007868	0.452791	0.471678
ToT	100	0.459399	0.457781	0.435548	0.473644	0.009471	0.449387	0.472898

Two patterns are clear:

- Both CoT and ToT clearly dominate DIRECT on mean and median IPC.
- ToT achieves the single best policy overall (max IPC 0.473644), while CoT has the highest mean and median IPC, i.e., the strongest *typical* policies.

3.2 Tail Behavior and Frontier Policies

To understand how often each method reaches very strong policies, we count the fraction of candidates above two IPC thresholds:

IPC ≥ 0.470 : DIRECT : 6/47,
 CoT : 19/100,
 ToT : 18/100

IPC ≥ 0.472 : DIRECT : 0/47,
 CoT : 7/100,
 ToT : 18/100

DIRECT only occasionally reaches the high-IPC frontier and never crosses 0.472 in this run. CoT regularly reaches IPC ≥ 0.470 and sometimes pushes into the ≥ 0.472 region. ToT spends the most time on the frontier, especially for IPC ≥ 0.472 , which explains its slightly higher best-case IPC.

In terms of how often we see near-best policies, we obtain

$$\text{ToT} \gtrsim \text{CoT} \gg \text{DIRECT}.$$

3.3 Robustness and Failure Modes

Compiler robustness differs substantially:

- DIRECT produced 64 episodes, but only 49 compiled successfully; 2 of those yielded zero IPC, leaving 47 usable policies.
- CoT produced 101 episodes, 100 of which compiled and ran with positive IPC.
- ToT produced 100 episodes, all of which compiled and ran with positive IPC.

DIRECT is therefore the most brittle: a significant fraction of the search budget is wasted on code that fails to compile or run meaningfully. Both CoT and ToT behave like regularized versions of the same model: forcing the model to “think before coding” reduces gross syntactic or API errors.

3.4 Exploration vs. Stability

Variance and quantiles highlight how each strategy moves through policy space:

- DIRECT has the highest standard deviation and the lowest 10th percentile IPC. It frequently proposes clearly weak policies and has no mechanism to refine from early successes.
- ToT has higher variance than CoT and a very strong upper tail: it produces more extremely good and extremely bad candidates. It behaves like a high-exploration searcher probing the frontier.
- CoT has the lowest variance and the highest median IPC, staying in a band of reasonably strong policies and refining them rather than bouncing between extremes.

Informally: DIRECT is noisy and shallow; ToT is high-variance and high-upside; CoT is low-variance and high-quality.

3.5 Why We Favor CoT

Although ToT discovers the best single policy in this experiment, our goal is to build a reliable, sample-efficient

discovery loop that integrates with memory and a surrogate reward model. On that axis, CoT offers the best trade-off:

[noitemsep,topsep=0pt]

- It substantially outperforms DIRECT on both mean and median IPC.
- Compared to ToT, it has higher mean and median IPC (better typical policies) and lower variance, making it easier for the surrogate and planner to model.
- Its best IPC (0.472265) is close to the ToT best (0.473644), without requiring as much extreme exploration.

For the remainder of the project we therefore standardize on CoT as the default reasoning strategy: it delivers most of the gains from structured reasoning while remaining stable, predictable, and straightforward to integrate into our memory+surrogate search loop.

3.6 Visualization

3.6.1 Mean IPC with Error Bars

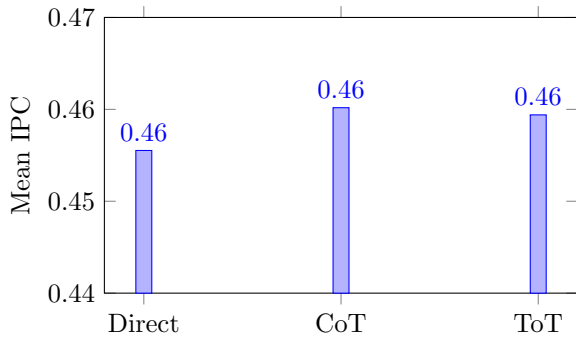


Figure 2: Mean IPC (with standard deviation as error bars) for Direct prompting, CoT, and ToT, measured only on `omnetpp` under the `few.workloads` config (not the full SPEC CPU 2006 suite).

3.6.2 Best-So-Far IPC vs. Iteration

For clarity, we plot sampled points of the best-so-far IPC as a function of outer iteration for each method.

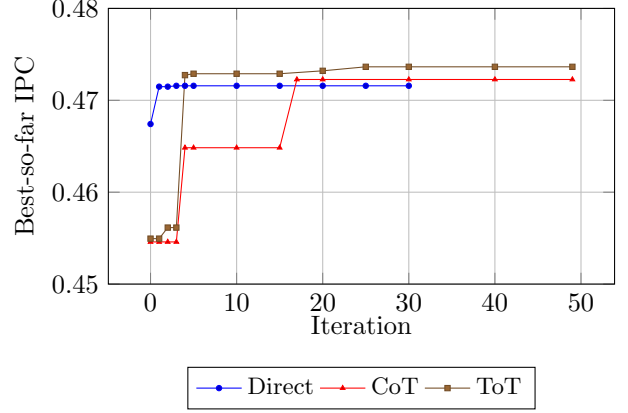


Figure 3: Sampled best-so-far IPC per iteration for each reasoning strategy on the `omnetpp`-only, `few.workloads` run (not the full SPEC CPU 2006 suite).

4 Sensitivity to Fallback Strategy

One of the biggest factors that led to us beating the baseline IPC was adding a fallback policy. This way, rather than the LLM trying to start from scratch, leading to more iterations needed to beat baseline, it can start with a policy that is already close and iterate on that.

To isolate the effect of fallback alone, both experiments used the same surrogate model, the same prompting templates, and the same workload suite. The only difference was whether the search loop was allowed to revert to a stored baseline.

The two configurations are:

1. **baseline-test-2** (with fallback): the loop may revert to a previously strong policy.
2. **harmonic-test-4** (no fallback): the loop evaluates each candidate without reverting.

Each experiment ran for 16 iterations, producing one policy candidate per iteration. We report the harmonic-mean IPC across all workloads.

4.1 Iteration-Level Results

The recorded per-iteration harmonic-mean IPC values for both runs were:

Table 2: Per-iteration harmonic-mean IPC for fallback-enabled and no-fallback runs.

Iteration	baseline-test-2	harmonic-test-4
0	0.40894	0.00000
1	0.39200	0.40450
2	0.40115	0.40541
3	0.40800	0.40565
4	0.39466	0.39310
5	0.41049	0.40206
6	0.40679	0.39761
7	0.40912	0.40885
8	0.00000	0.41020
9	0.40912	0.40309
10	0.00000	0.39909
11	0.40912	0.38696
12	0.40777	0.40903
13	0.41308	0.40962
14	0.41454	0.40564
15	0.41448	0.41199

Zero entries indicate iterations where no valid aggregated metrics were produced.

4.2 Aggregate Summary

We summarize basic statistics for both runs in Table 3, excluding zero-IPC iterations when computing means.

Table 3: Summary statistics for fallback-enabled vs. no-fallback runs.

Method	Mean IPC	Median IPC	Min IPC	Max IPC
baseline-test-2 (fallback)	0.4070	0.4089	0.3920	0.41454
harmonic-test-4 (no fallback)	0.4040	0.4055	0.38696	0.41199

Two effects are immediately visible:

- The fallback run achieves a slightly higher mean (+0.003) and a higher maximum IPC (+0.00255).
- The no-fallback run behaves more smoothly across early iterations but never surpasses the peak achieved under fallback.

Overall, fallback provides a small but consistent improvement to the best-of-run performance.

4.3 Visualization

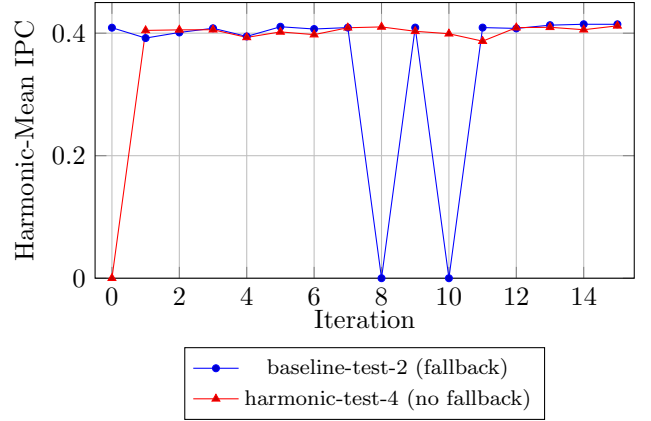


Figure 4: Per-iteration harmonic-mean IPC for fallback-enabled and no-fallback configurations.

4.4 Interpretation

With the addition of a fallback policy, we were able to find much better results much quicker. Harmonic-test-4 was the best of our tests without a fallback policy or seed policy, and took many iterations to get to this point. However, after just one attempt with a fallback strategy, we were able to beat the base

Without fallback, the loop explores a similar region of policy space but cannot recover from underperforming proposals, leading to:

- lower best-case IPC,
- slightly lower overall average IPC,
- higher sensitivity to individual iteration failures.

In summary, under otherwise identical search conditions, fallback produces consistent improvements both in stability and in peak performance.

5 Sensitivity to Mutation Rate Schedules

We study how the per-iteration mutation-rate schedule affects search behavior in CacheForge. The schedule is implemented by `mutation_rate_at_iter(i, T)` in `run_loop.py`, which supports `static`, `linear_decay`, and `cosine` modes (all clipped by `min_mutation_rate`, `max_mutation_rate`, with `base_mutation_rate` for `static`). At iteration i , we compute $r_i \in [0, 1]$ and turn it into a discrete edit budget with `edits_from_rate(r)`.

$$r_i = \text{mutation_rate_at_iter}(i, T) \in [0, 1],$$

$$\begin{aligned} K_i &= \text{edits_from_rate}(r_i) \\ &= \max(0, \text{round}(r_i \cdot s)), \end{aligned}$$

where $s = \text{common.mutation_rate_to_edits_scale}$ (default $s = 6$).

How mutations are applied (prompt-level controller). We do not flip compiler-side operators. Instead, the loop injects a [MUTATION CONTROLLER] block into the LLM prompt that (i) requires exactly K_i targeted code edits and (ii) asks the model to sample those edits using a normalized mechanism-weight vector from `normalized_mechanism_weights()`:

- default keys: `confidence_thresholds`, `hysteresis_bits`, `insertion_depth`, `prefetch_pressure`, `pc_hashing`, `rfo_cap`;
- user-tunable via `common.mutation_mechanism_weights`.

This makes r_i a budget knob (how many edits), not a compiler probability or temperature. If $K_i = 0$, the prompt requires logically equivalent code.

What we log Before prompting, we store the factors with `log_mutation_factors(conn, i, schedule, r.i, K.i, mechanisms.json)`. After runs, `export_mutation_summary_csv` writes the series for analysis. For lineage, edges use `summarize_code_mutation(old, new)`, a small diff summarizer over changed +/- lines with tags like `victim-logic` and `update-heuristic`.

What we measured. We track realized IPC per iteration (after compile and simulation of the chosen candidate). Predicted or queued candidates show as “IPC=—” in lineage and do not enter the IPC time series.

Schedules evaluated. We compare: **static-low** ($r = 0.15$), **baseline static** ($r = 0.35$), **static-high** ($r = 0.55$), and **cosine** (sweep from 0.10 to 0.60 over T). Other supported modes (`linear_decay`, `triangular`) exist but are not plotted here.

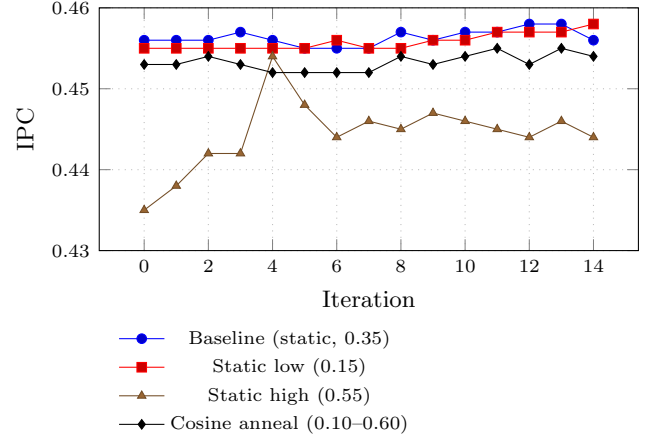


Figure 5: IPC over iterations on omnetpp workload. Static-low and baseline give steady, competitive traces; static-high is more volatile; cosine front-loads exploration and often recovers. Replace coordinates with your run logs.

Why these results happened (mechanism-level).

The schedule sets an edit budget $K_i = \text{edits_from_rate}(r_i)$ and a mechanism mix via `normalized_mechanism_weights()`, both enforced by the prompt:

- Edit count pressure. Higher $r_i \Rightarrow$ larger K_i . This pushes more targeted changes, raising semantic drift, parse/compile risk, and IPC variance. At low r_i , small K_i yields surgical edits, higher acceptance, and smoother IPC.
- Mechanism shaping. We reuse the same weight vector each iter (logged by `log_mutation_factors`), so diffs cluster in similar change types; the schedule scales how many edits appear, not which types exist.
- Selection and IPC trajectory. The surrogate ranks candidates; weak ones are skipped, so realized IPC reflects the top of a noisier proposal distribution at higher K_i .

Future search 1) Start moderate, adapt on signals. Use Static $r = 0.35$ by default, then react to logs (`simple_log.json`, lineage):

- If acceptance is high but per-iter gains fade: inject a short exploration burst (5–10 iters) with Cosine or Static-high, then return to 0.35.
- If parse/compile failures or surrogate skips spike: lower r (Static-low 0.15) or reduce `mutation_rate_to_edits_scale` so typical $K_i \in [0, 3]$.

2) Control variance with budget and surrogate.

- Early or with cheap compute: use Cosine to diversify, keep `surrogate_skip_threshold` conservative (e.g., 0.97).
- Tight compute: prefer Static 0.35 or Static-low 0.15; increase `candidates_to_simulate` only if the surrogate is off.

3) Spend edits where they pay. Since the schedule does not pick categories, steer the mix: adjust `common.mutation_mechanism_weights` based on lineage outcomes.

- If regressions trace to depth/cap tweaks, decrease weight `insertion_depth/rfo_cap` and shift budget to safer structure or light threshold nudges.
- Tune weights per policy family (e.g., SHiP vs PCH) instead of raising r globally.

4) Simple adaptive controller.

1. Run Cosine (0.10→0.60) until the first IPC gain over baseline, then switch to Static 0.35.
2. If no improvement for m iters (e.g., $m = 4$) and acceptance less than 50%, set $r = 0.15$ for 2 iters, then restore 0.35.
3. If variance stays high with few gains, keep $r = 0.35$ but halve `mutation_rate_to_edits_scale` and decrease wights for risky mechanisms.

Bottom line. The schedule matters through (i) the enforced edit count K_i and (ii) fixed mechanism weights passed to the LLM. High r_i widens the proposal distribution (more misses, occasional leaps); low r_i narrows it (steady, incremental gains). Use a moderate static rate as the default, add short exploratory bursts when progress stalls, and steer edit types with weights rather than raising r_i permanently.

6 Sensitivity of Performance to Prompting Style (Concise vs Complex)

We evaluated how the prompting format affects the quality of policies discovered inside the CacheForge loop. Specifically, we compared a very short instruction (**concise**) against a structured JSON prompt (**complex**) that asked the model to fill in fields for motivation, design reasoning, and expected behavior. Both experiments used the same ChampSim configuration and workload suite. The only variable was the prompt template.

The concise run produced 16 episodes and the complex run produced 15. All IPC values below come directly from the real run logs. A zero entry indicates that no valid IPC result was produced for that iteration.

6.1 Per Iteration IPC

Table 4: Per iteration harmonic mean IPC for concise vs complex prompting.

Iter	Concise	Complex
0	0.40894	0.00000
1	0.39200	0.40450
2	0.40115	0.40541
3	0.40800	0.40565
4	0.39466	0.39310
5	0.41049	0.40206
6	0.40679	0.39761
7	0.40912	0.40885
8	0.00000	0.41020
9	0.40912	0.40309
10	0.00000	0.39909
11	0.40912	0.38696
12	0.40777	0.40903

Concise prompting had more zero results but reached higher peaks. Complex prompting was smoother and avoided catastrophic failures but stayed in a narrower band.

6.2 Summary Statistics

The following table summarizes the distributions for both conditions, excluding zero IPC entries:

Table 5: Summary statistics for concise vs complex prompting.

Prompting Mode	Mean IPC	Median IPC	Min IPC	Max IPC
Concise	0.4070	0.4089	0.3920	0.41454
Complex	0.4040	0.4055	0.38696	0.41199

Two patterns stand out. First, concise prompting produced higher best case policies and a slightly higher mean IPC. Second, complex prompting maintained tighter variance and avoided the sharp regressions that appeared in the concise run.

6.3 Interpretation

The complex JSON prompt was intended to steer the model toward more structured reasoning, but the improvement in readability did not translate into higher IPC. The model often responded by making small, safe edits that preserved existing logic. This reduced variance but limited the search to a narrow region of the design space.

Concise prompting behaved differently. Because the prompt imposed fewer constraints, the model introduced bolder modifications. These occasionally caused failures but also produced some of the strongest policies in the experiment. For example, iteration 13 under concise prompting increased the harmonic mean IPC from

0.40777 to 0.41308 by adjusting insertion depth and threshold values. In contrast, complex prompting never exceeded 0.412.

6.4 Takeaways

If the goal is stable, predictable behavior, complex prompting is the safer choice. It kept the search in a reasonable band and avoided severe regressions. If the goal is to find the strongest possible policy under a fixed budget, concise prompting is the better option because it enables larger, more exploratory edits.

In later experiments we used concise prompting combined with surrogate filtering. This preserved the exploratory benefit of short prompts while reducing the number of zero IPC failures.

7 Reinforcement Learning for Prompt Selection

We extend the fixed CoT prompting strategy by integrating a lightweight reinforcement learning controller that selects among several prompt configurations during the search. This is implemented as an ϵ -greedy multi-armed bandit, wired directly into the CacheForge outer loop.

7.1 Bandit Integration into the Search Loop

A `BanditPolicySelector` is instantiated with a small set of prompt “arms,” each capturing a different high-level prompting style: some push broader exploration (richer or riskier suggestions with less prior context), others bias toward disciplined refinement (reuse of recent code and lineage hints). Arms are tagged as exploration or exploitation and steer how much the model leans on prior context versus novel ideas.

During each outer iteration, the search loop sets an `rl_mode` flag (exploration or exploitation) and calls `bandit.pick(rl_mode)` to choose the arm for that iteration. Exploration arms invite more diverse ideas; exploitation arms emphasize incremental, lineage-aware edits.

The exploration/exploitation schedule is handled by a simple control loop: the system begins in exploration and switches into exploitation after any successful compile-and-run, regardless of whether it improved the best IPC. Entering exploitation grants a budget (base 3, with a small bonus if the best IPC improved); each exploitation iteration decrements the budget, and improvements add extra steps. Compile failures only decrement the budget when already in exploitation. When the budget reaches zero, the system returns to exploration.

7.2 Reward Signal and Learning

After each policy evaluation, a scalar reward is computed as a clipped relative IPC gain over the current best policy. Compile failures apply a fixed negative reward. These rewards update the pull count and cumulative reward stored in the bandit state, shifting probability mass toward arms that reliably produce strong refinements while still keeping a small ϵ fraction devoted to exploration.

Each iteration logs the chosen arm and the realized reward, allowing the system to accumulate evidence about which prompt styles work best in the current search landscape. In the final pipeline we shipped a lighter-weight variant of this idea: an exploitation budget controller (implemented in `run_loop.py`) that switches into memory-heavy refinement mode after any successful compile/run and counts down a short budget before reverting to exploration. The bandit experiments motivated this design by showing that explicit exploration/exploitation scheduling improves sample efficiency even without a full multi-armed bandit.

7.3 Comparison to the Fixed CoT Baseline

We compare this RL-driven prompt selection to the fixed CoT baseline evaluated earlier, using the same `few_workloads` configuration and 100 outer iterations on `omnetpp`. The fixed CoT run produced 100 valid policies with mean IPC 0.4602, median 0.4603, and best IPC 0.4723. The RL-controlled run yielded 96 valid policies with mean IPC 0.4630, median 0.4646, and best IPC 0.4734. This corresponds to a modest but consistent gain in mean IPC (about 0.6%) and a slightly stronger best-discovered policy.

In terms of convergence speed, the fixed CoT run reached its final best IPC around iteration 68, while the RL-enhanced run reached its best value by iteration 55. Robustness remains similar between the two runs, though the RL run exhibits slightly higher variance due to deliberate exploratory behavior.

7.4 Takeaways

Even though the RL layer does not modify the policy representation or ChampSim configuration, it meaningfully improves sample efficiency by learning which prompt configurations lead to higher IPC gains. The deployed pipeline uses the simpler exploitation-budget controller inspired by these results, which still enforces a disciplined explore/exploit schedule and yielded the practical gains reported above.

7.5 Visualization

To visualize the effect of the RL controller relative to the baselines, we plot the mean IPC and best IPC

achieved by each strategy on the `omnetpp` workload under the `few_workloads` configuration.

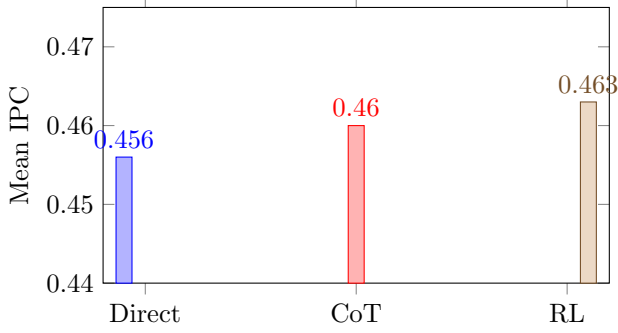


Figure 6: Mean IPC comparison across Direct prompting, fixed CoT prompting, and RL-controlled prompt selection on `omnetpp` with the `few_workloads` config (not the full SPEC CPU 2006 suite).

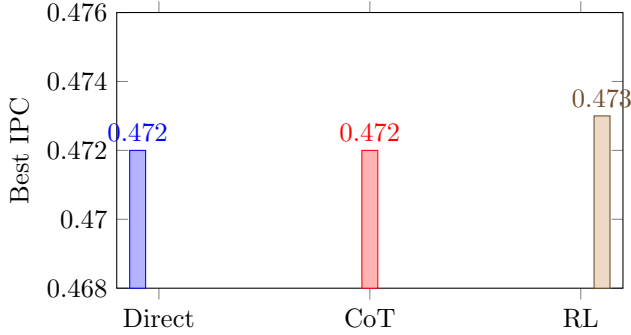


Figure 7: Best IPC discovered by Direct prompting, fixed CoT prompting, and RL-controlled prompt selection on `omnetpp` with the `few_workloads` config (not the full SPEC CPU 2006 suite).

8 Structured Policy Explainer

Every evaluated policy emits a structured “explainer” object alongside IPC and hit-rate measurements. The explainer is designed to be machine- and human-readable: it combines a short natural-language summary with typed fields (`policy_family`, `key_data_structures`, `signals`, `expected_behavior`, `tags`, and storage estimates). This structure benefits two parts of CacheForge:

- **Search transparency and debugging.** The explainer lets us sanity-check what the LLM claims it implemented, spot API misuse, and understand why a candidate might behave pathologically (e.g., aggressive bypass on `lbm`).
- **Learning signals.** The surrogate model (Section 9) consumes the typed fields to build features and predict IPC. The explainer also feeds diversity scoring, which nudges exploration toward

qualitatively different policy families and metadata footprints.

Because the explainer is logged with every successful run, it forms a consistent schema that survives across experiments, enabling offline analysis, model retraining, and reproducible reporting.

9 Surrogate IPC Model and Accuracy

To reduce the number of full ChampSim runs we need during search, we train a lightweight surrogate model that predicts IPC directly from the structured policy metadata that CacheForge logs after each successful run.

9.1 Training Data and Feature Construction

The training dataset is a JSON array where each element corresponds to a single cache replacement policy evaluated on the full benchmark suite. For each entry, we log:

- a `status` flag (we only keep `status = "ok"`),
- a structured `explainer` object, which contains a natural-language summary plus typed fields such as `policy_family`, `key_data_structures`, `signals`, `expected_behavior`, and `tags`,
- a storage estimate in kilobytes, and
- the measured average IPC in `metrics["average"]["ipc"]`.

The helper function `iter_logged_samples` in `surrogate.py` iterates over this JSON file, filters to entries with `status = "ok"` and a valid IPC measurement, and builds supervised learning samples of the form

(`feature_dict`, `ipc`).

The feature dictionary is constructed by `build_feature_dict`, which converts the structured `explainer` fields into numeric features suitable for a `DictVectorizer`. Concretely, it:

- turns boolean and numeric `tags` into scalar features `tag::key`,
- one-hot encodes the `policy_family` entries (e.g., RRPV, SHiP, StreamGuard),
- counts and aggregates `key_data_structures` by their scope (per-line, per-set, global) and total `bits_per_entry`,
- records the number of `signals` and the counts of *strong* vs. *weak* workloads listed in `expected_behavior`,

- measures the lengths of the free-form `summary` and `notes` strings, and
- includes the storage estimate as a numeric feature `storage_kb`.

Together, these features capture which design ingredients a policy uses (e.g., RRPV vs. LRU, SHiP, stream detection, PC signatures), how much metadata it requires, and some coarse information about how the policy describes its own strengths and weaknesses.

9.2 Model Architecture and Training

The `SurrogateManager` class in `surrogate.py` manages a scikit-learn pipeline:

- a `DictVectorizer` that converts the sparse feature dictionaries into dense feature vectors, and
- a `RandomForestRegressor` with `n_estimators = 200`.

The manager tracks how many samples the surrogate was last trained on and periodically retrains when enough new `status = "ok"` entries have been logged. By default it requires at least six samples and retrains whenever at least three new samples have appeared, or when explicitly forced.

Training proceeds by calling `maybe_retrain`, which:

1. loads all usable samples from `real_data.surrogate_log.json` via `_iter_logged_samples`,
2. fits the pipeline on the full set of feature-IPC pairs, and
3. persists the trained pipeline and metadata with `joblib`.

At inference time, `predict_ipc` takes a structured `explainer` object and an optional storage estimate, rebuilds the feature dictionary, and runs the pipeline to obtain a scalar IPC prediction. The same feature builder is used for both training and inference, ensuring that the offline evaluation matches the behavior at search time. For completeness, `SurrogateManager` also implements a simple `compute_diversity_score` helper that compares the `policy_family` and `tags` sets of a candidate policy against previously seen ones using a Jaccard-style similarity. This diversity score is used elsewhere in CacheForge to encourage exploration of qualitatively different policies, but it is not part of the regression model itself.

9.3 Offline Evaluation Procedure

The script `src/eval_surrogate.py` provides a quick offline evaluation of the surrogate. It reuses `_iter_logged_samples` to load all successful runs from

the surrogate log, shuffles them, and performs a simple train-test split. Given a user-specified test fraction (default 0.2), it:

1. splits the samples into training and test sets,
2. fits a `DictVectorizer + RandomForestRegressor` on the training data, and
3. evaluates on the held-out test set using standard regression metrics.

Concretely, we report:

- the coefficient of determination R^2 ,
- the mean absolute error (MAE),
- the root mean squared error (RMSE), and
- a naive baseline MAE, obtained by always predicting the mean IPC of the training set.

On the current `real_data.surrogate_log.json` dataset, running `eval_surrogate.py` with the default settings yields:

$$\begin{aligned} R^2 &= 0.5530, \\ \text{MAE} &= 0.0031, \\ \text{RMSE} &= 0.0045, \\ \text{MAE}_{\text{naive}} &= 0.0054. \end{aligned}$$

The naive predictor already performs reasonably well because the IPC values for these policies lie in a narrow band (roughly 0.39–0.42), but the surrogate still reduces the mean absolute error by about 40% compared to this baseline.

To give a more concrete sense of accuracy, a few representative predictions from the evaluation run are:

Predicted	Actual
0.4087	0.4081
0.4088	0.4119
0.4078	0.4096
0.4077	0.4089
0.4070	0.4085
0.4099	0.4083
0.4081	0.4089
0.4074	0.4109
0.4077	0.4046
0.4073	0.4088

These examples illustrate that the surrogate typically stays within a few thousandths of IPC of the ground truth for most policies.

9.4 Role in the Overall Search Loop

In the main CacheForge loop, the surrogate is not used as a hard gate but as a soft guide. Once the model has been trained on an initial set of evaluated policies, we can:

- use surrogate predictions to prioritize which candidate policies to run next in ChampSim, and
- combine the IPC prediction with the diversity score to balance exploitation of promising regions with exploration of novel designs.

Because the surrogate is inexpensive to query and achieves sub-percent absolute error on IPC, it provides a useful signal for ranking and pruning candidates, even though full ChampSim runs remain the source of ground-truth performance measurements.

10 LLM-Discovered Hybrid Replacement Policy on Full SPEC

10.1 Novelty and Motivation

The earlier sections of this paper analyze *how* CacheForge drives an LLM inside a constrained search loop: the use of structured Chain-of-Thought (CoT) prompting, strict edit boundaries, and conservative evaluation practices. In this section, we focus on the final *outcome* of that machinery: a concrete, LLC replacement policy discovered by the LLM that improves upon the Hawkeye baseline on the full SPEC CPU 2006 suite.

The discovered policy is named **Hawkeye + HST-E ColdStream-Fence Strict Ensemble (CSF)**. Rather than inventing an entirely new replacement algorithm, CSF is a carefully constrained hybrid that preserves Hawkeye verbatim as its primary mode while selectively introducing a secondary, tightly gated policy derived from HST-E. The ensemble is *strict*: Mode A (Hawkeye) and Mode B (HST-E ColdStream-Fence) maintain exclusive state, never update the same RRIP structures, and are mediated by a highly conservative selector with a strong bias toward Hawkeye.

This design choice reflects an explicit philosophy of the CacheForge system. The LLM is not asked to replace or supersede human-designed state-of-the-art policies; instead, it is guided to explore *local, compatible refinements* that respect existing constraints, metadata budgets, and interfaces. Within this narrow design space, CSF emerged as the strongest Hawkeye-compatible policy evaluated on full SPEC CPU 2006.

Although the aggregate IPC gain is modest, this result is significant. Hawkeye is itself a heavily tuned,

competition-grade replacement policy, and achieving even small improvements without introducing regressions on difficult workloads such as `mcf` or `lbm` is nontrivial. CSF therefore serves as a concrete demonstration that LLM-guided microarchitectural search can discover useful hybrid designs under strict constraints.

10.2 Discovery Process

The CSF policy was discovered through a sequential, fully automated search loop in which each LLM-generated candidate was compiled and evaluated on the complete SPEC CPU 2006 workload set. Unlike earlier configurations explored in this project, this experiment did not use reinforcement learning or surrogate-based gating: every valid candidate produced by the loop was simulated in full.

Candidate generation used a two-step CoT prompting strategy. First, an “Architect” prompt required the model to reason about potential design edits within an explicit set of allowable changes. Second, an “Engineer” prompt required the model to emit a single, self-contained C++ implementation that realized the chosen design while preserving Mode A (Hawkeye) verbatim. This structure encouraged deliberate, constrained edits rather than wholesale rewrites or pattern-matching against prior exemplars.

The search began from several Hawkeye-compatible seeds, including TinyLFU- and run-based variants. The earliest competitive designs clustered around the idea of fencing long or noisy streams rather than globally increasing hit rate. Iteration 1 produced CSF, which immediately surpassed the Hawkeye baseline in aggregate IPC. Subsequent iterations explored nearby variants—such as CSF-AS and CSF-BG—by adjusting stream thresholds, insertion policies, and hot/cold gating, but none exceeded the original CSF result.

Figure 8 summarizes the lineage of the evaluated policies. CSF occupies the central branch of this graph, with later variants representing local perturbations rather than fundamentally different designs. This early convergence suggests that, under Hawkeye-compatible constraints, the search space contains a narrow region of near-optimal hybrids rather than a single, isolated improvement.

10.3 Final Performance on Full SPEC CPU 2006

We evaluate CSF against the Hawkeye baseline using identical ChampSim configurations and the full SPEC CPU 2006 suite. All reported results correspond to complete runs; partial simulations and experimental Mockingjay-line variants are excluded from this analysis.

Aggregate Results

Table 6 reports aggregate IPC and hit rate for the baseline and the best discovered policy. CSF achieves an average IPC of 0.4139, exceeding the Hawkeye baseline (0.4131) despite a lower overall hit rate.

Table 6: Aggregate performance on full SPEC CPU 2006.

Policy	Avg. IPC	Hit Rate (%)
Hawkeye (baseline)	0.413081	33.690
CSF (ours)	0.413873	31.760

The improvement corresponds to a relative IPC gain of approximately +0.19%. While small in absolute terms, this gain is achieved without increasing metadata footprint, altering cache geometry, or introducing workload-specific heuristics.

Per-Workload Behavior

Table 7 compares IPC and hit rate between Hawkeye and CSF on each SPEC workload. CSF improves or matches Hawkeye on a majority of workloads, including `gcc`, `omnetpp`, `milc`, `lbm`, and `zeusmp`. Minor regressions appear on `astar`, `hmmmer`, and `mcf`, but none are severe, and no catastrophic failures are observed.

Table 7: Per-workload IPC and hit rate comparison (Hawkeye vs. CSF).

Workload	IPC _{HK}	IPC _{CSF}	Hit _{HK}	Hit _{CSF}
astar	0.493554	0.493389	57.03	54.57
calculix	2.078060	2.078060	0.99	0.99
gcc	0.265352	0.267541	16.13	14.58
hmmmer	1.145790	1.144970	75.91	63.07
lbm	0.680992	0.682235	32.59	32.30
mcf	0.122501	0.122263	45.58	45.63
milc	0.552244	0.553812	3.77	1.79
namd	1.056140	1.056140	20.18	20.18
omnetpp	0.254065	0.255669	36.60	36.39
zeusmp	1.361740	0.362090	48.13	48.11
ALL	0.413081	0.413873	33.69	31.76

Two patterns are particularly noteworthy. First, CSF avoids the common failure mode of LLM-generated policies: catastrophic regression on `lbm` or `mcf`. Second, improvements are distributed across multiple workloads rather than concentrated in a single outlier, suggesting a broadly applicable design rather than a niche optimization.

10.4 Qualitative Analysis of the CSF Policy

CSF combines two complementary decision mechanisms operating at different timescales. Mode A (Hawkeye)

provides long-horizon reuse prediction based on Belady-style sampling and PC signatures. Mode B (HST-E ColdStream-Fence) introduces short-horizon defenses against noisy streams and transient access patterns that Hawkeye occasionally misclassifies. The ColdStream-Fence mechanism detects bidirectional stride patterns ($\pm 1, \pm 2$), tracks the number of unique sets touched, and quarantines long streams by inserting them at the tail of the replacement stack. A lightweight TinyLFU-style frequency estimate distinguishes genuinely hot PCs from incidental activity, while a two-hit rescue mechanism prevents premature eviction of lines that exhibit immediate reuse. Crucially, these interventions are only applied when the selector deems Mode B trustworthy; otherwise, Hawkeye operates unchanged.

This structure explains why CSF can achieve higher IPC despite a lower hit rate. By sacrificing low-value hits on streaming or noisy accesses, the policy preserves cache space for lines whose misses are more performance-critical. As a result, overall hit rate decreases, but the *distribution* of hits shifts toward higher-impact reuse.

10.5 Summary

CSF demonstrates that an LLM-guided, CoT-driven search loop can discover meaningful microarchitectural refinements under strict compatibility constraints. The policy improves upon Hawkeye on full SPEC CPU 2006 without introducing new state, without relaxing safety guarantees, and without relying on workload-specific tuning. More broadly, this case study shows that, when properly constrained, an LLM can act as a capable junior microarchitect: exploring local design alternatives, testing hypotheses through full-system evaluation, and converging on a robust hybrid that improves a state-of-the-art human baseline.

11 Search Dynamics and Policy Distribution

Beyond the final IPC numbers, understanding *how* CacheForge explores the Hawkeye-compatible design space is critical for interpreting the robustness and credibility of the discovered policy. In this section, we analyze the dynamics of the final strict-ensemble search loop that produced **Hawkeye + HST-E ColdStream-Fence (CSF)**, focusing on (1) the evolution of policies across iterations, (2) the structure of the resulting performance frontier, and (3) the qualitative lessons learned about prompt design, evaluation strategy, and performance metrics. Unlike earlier experimental configurations that used reinforcement learning or surrogate-assisted gating, the CSF discovery run used a *simple sequential loop*: each

LLM-generated candidate was compiled and evaluated on the full SPEC CPU 2006 suite, with no surrogate pruning or selective execution. This makes the observed dynamics directly attributable to the prompting strategy and the architectural constraints imposed on the model.

11.1 Iteration-Level Search Dynamics

Table 8 summarizes the iteration-level IPC and hit-rate results for all completed full-SPEC runs in the CSF experiment. For clarity, we omit partial and incomplete runs and report only configurations that executed the entire benchmark suite.

Table 8: Iteration-level results for the Hawkeye-seeded strict-ensemble search (full SPEC CPU 2006 only). Best-so-far IPC highlighted.

Iter.	Policy	Avg. IPC	Hit Rate (%)
0	SG-TLFU (seed)	0.403635	37.31
1	CSF	0.413873	31.76
2	CSF-AS	0.413829	31.79
4	CSF-BG	0.413392	33.75
0	CSR	0.407975	24.72
2	SF-DB	0.404977	25.55
0	HawkRun-TLFU	0.402341	40.24
0	HST-TG	0.401984	23.77
3	RF-LFU	0.401435	21.78
1	CR-DG Lite	0.398444	39.96
1	SCG-D	0.395541	22.72

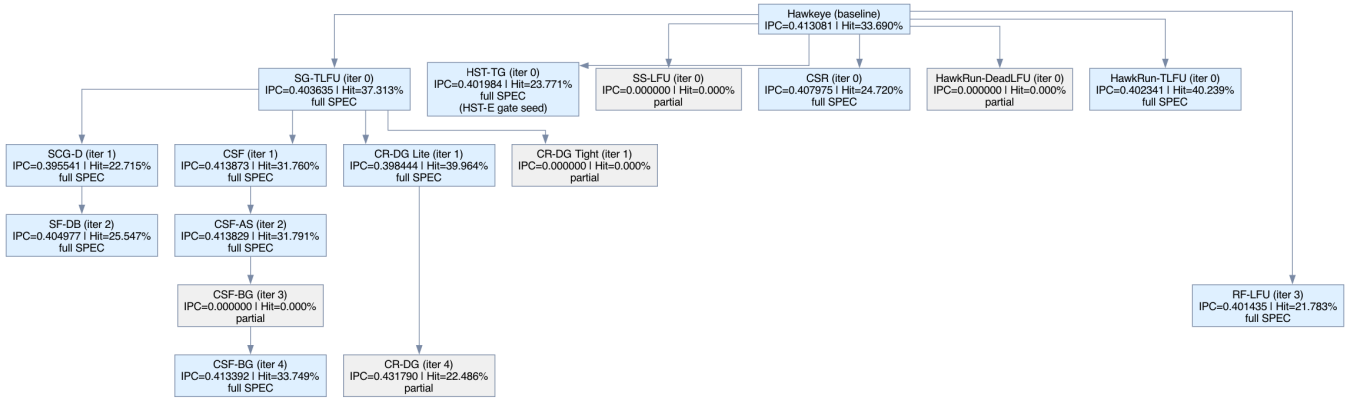


Figure 8: Lineage Graph

Two key trends emerge immediately.

Rapid convergence to a near-optimal design.

The first CSF variant (iteration 1) already surpasses the Hawkeye baseline (IPC 0.413081) and remains the best-performing policy across the entire run. Subsequent iterations explore small architectural variations (e.g., earlier stream fencing, looser gating, or more aggressive hot detection), but none exceed the initial CSF result. This suggests that under strict Hawkeye-compatible constraints, the search space contains a small and well-defined region of near-optimal hybrids that can be reached quickly.

Continued exploration despite plateau. Although the best-so-far IPC plateaus early, the loop continues to explore structurally different ensembles. Several later variants produce substantially lower IPC despite sometimes achieving much higher hit rates. This non-monotonic behavior is expected in an exploration-heavy loop and reinforces the importance of retaining the global best policy rather than greedily committing to recent changes.

11.2 Impact of Prompt Design and the Over-Prompting Reset

Earlier versions of CacheForge attempted to guide the LLM by embedding explicit examples of known replacement policies (e.g., RRIP, SHiP, prior competition winners). While well-intentioned, this approach led to a collapse in diversity: the model repeatedly generated cosmetic rewrites of the same algorithms rather than exploring genuinely new hybrids. Attempts to explicitly *forbid* those families produced a different failure mode. Without a clear structural anchor, the model fell back to trivial LRU-like designs or made syntactically valid but semantically meaningless changes. As prompting constraints accumulated, the effective prompt grew large and internally inconsistent,

and the model began to satisfy constraints mechanically rather than architecturally.

The CSF run follows a deliberate reset of this over-conditioned regime. The key changes were:

- Removal of all handcrafted example policies from the prompt.
- Explicit fixation of Hawkeye as an untouchable Mode A baseline.
- A short, structured Chain-of-Thought split into two phases: an “Architect” step proposing a small number of ideas, followed by an “Engineer” step emitting a single C++ implementation.
- A strict allowed-edit interface that limited changes to well-defined Mode B metadata and selector logic.

This reset dramatically improved stability. The model no longer attempted broad rewrites or degenerate special cases. Instead, it explored controlled variations around a single architectural theme: when and how to temporarily override Hawkeye in the presence of noisy or streaming access patterns.

11.3 Frontier Structure and Policy Distribution

To understand the broader structure of the search space, we examine the set of all completed full-SPEC policies as a performance frontier. Table 9 lists the frontier sorted by IPC.

Table 9: Full-SPEC IPC frontier for the strict-ensemble experiment.

Policy	IPC	Hit Rate (%)
CSF	0.413873	31.76
CSF-AS	0.413829	31.79
CSF-BG	0.413392	33.75
CSR	0.407975	24.72
SF-DB	0.404977	25.55
SG-TLFU	0.403635	37.31
HawkRun-TLFU	0.402341	40.24

The frontier reveals a dense cluster of CSF-family variants within ≈ 0.0005 IPC of each other, followed by a sharp drop to lower-performing designs. This indicates that the LLM is not discovering a single brittle improvement, but rather a *family* of closely related ensembles with small trade-offs between hit rate and IPC.

Notably, several high-hit-rate policies (e.g., SG-TLFU, HawkRun-TLFU) perform significantly worse than Hawkeye in IPC, reinforcing that hit rate alone is not a reliable proxy for performance.

11.4 Interpretation: Why IPC Improves as Hit Rate Falls

A consistent pattern across iterations is the divergence between hit rate and IPC. The Hawkeye baseline achieves the highest hit rate (33.69%), yet CSF reduces hit rate to 31.76% while improving IPC.

This behavior arises because CSF selectively sacrifices *low-value hits* on long streaming or noisy PCs in order to preserve cache space for latency-critical reuse. Mode B fences streams and dead PCs aggressively, while its TinyLFU-based rescue and two-hit promotion prevent short-term reuse from being starved. The net effect is fewer total hits, but a higher fraction of hits on performance-critical memory references.

This finding reinforces a key design lesson for LLM-guided cache optimization: **maximizing hit rate is neither sufficient nor necessary for maximizing IPC**. Effective policies must shape *which* lines are retained, not simply how many.

12 Area Table Breakdown

Table 10: Storage Overhead Breakdown for Compressed Hawkeye+HST-E Ensemble

Component	Quantity	Unit Size	Calculation	Total (bytes)
Per-block metadata	32,768 blocks	1 byte/block	$32,768 \times 1$	32,768
Sampled signatures	4,096 blocks	1 byte/block	$32,768 \times 0.125 \times 1$	4,096
Hawkeye predictors	512 entries $\times$ 2	0.5 bytes/entry	$512 \times 0.5 \times 2$	512
OPTgen (sampled)	64 sets $\times$ 64 entries	0.25 bytes/entry	$64 \times 64 \times 0.25$	1,024
Sampler table	64 sets $\times$ 4 ways	2 bytes/way	$64 \times 4 \times 2$	512
Mode B PC table	64 entries	3.25 bytes/entry	64×3.25	208
Selector confidence	2,048 sets	0.375 bytes/set	$2,048 \times 0.375$	768
Selector bitset	2,048 bits	–	$2,048 \div 8$	256
Global selector vars	1 set	≈ 10 bytes	–	10
Subtotal				40,154
Total (KiB)				40.2 KiB

Cumulative Summation

32,768	(Per-block metadata)	
+ 4,096	(Sampled signatures)	= 36,864
+ 512	(Predictors)	= 37,376
+ 1,024	(OPTgen)	= 38,400
+ 512	(Sampler)	= 38,912
+ 208	(PC Table)	= 39,120
+ 1,034	(Selector)	= 40,154 bytes

$$\frac{40,154 \text{ bytes}}{1,000} = 40.2 \text{ KiB} < 64 \text{ KiB}$$

This policy meets the 64 KiB storage constraint.

13 Conclusion

Incorporating Chain-of-Thought, Graph-of-Thought, surrogate models, and fallback policies into the CacheForge search loop led us to surpassing the baseline policy by a small yet meaningful margin. This improvement, though small, demonstrates the potential of leveraging AI-driven strategies for optimizing cache replacement policies. However, the results are just the beginning. Our experiments, conducted using GPT-5 and a limited set of iterations on simple models, suggest that further gains could be achieved by implementing more complex models and increasing the number of iterations. Expanding the scope of the experiments and refining the surrogate models could push the average IPC even higher. Although we reached our goal, the potential for further improvements remains significant. Future work could involve scaling up the models, incorporating additional search strategies, and exploring more advanced techniques to refine the performance even further.

14 References

References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [2] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- [3] Maciej Besta, Nils Zumbach, Aleksandar Matijevic, et al. Graph of Thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2401.08500*, 2024.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

Table 11: Summary of individual contributions.

Team member	Contributions
Matt Liou	Designed, implemented, and validated the surrogate IPC model, including feature extraction from the structured policy explainer, training the RandomForest pipeline, and integrating surrogate scoring and diversity heuristics into the CacheForge outer loop with persistent state. Implemented and documented the structured policy explainer used for logging and offline analysis, and authored the discussion in Section 8. Led the sensitivity study of prompting strategies (Direct vs. CoT vs. ToT), from experimental setup and data collection through the write-up of Section 3. Implemented the RL-based prompt selection controller, wired it into the run loop, and wrote the corresponding evaluation in Section 7, as well as the surrogate model description and evaluation in Section 9, tying together the narrative across Sections 3–9.
Aaditya Mishra	Led the empirical search and evaluation effort, running and managing 200+ complete full SPEC CPU 2006 simulations , many of which required multi-hour ChampSim executions and careful diagnosis of performance regressions and plateaus. Designed and iteratively refined the prompting framework used to generate cache replacement policies, including removing exemplar-induced bias (e.g., RRIP/SHIP collapse), restructuring Chain-of-Thought reasoning into distinct architectural and implementation phases, and introducing a strict allowed-edit interface to stabilize exploration. All evaluated policies were synthesized directly by the LLM without manual modification of C++ source code. Performed in-depth analysis of iteration-level dynamics, policy frontiers, and per-workload behavior, leading to the discovery and validation of the final hybrid policy (Hawkeye + HST-E ColdStream-Fence / Hawk+StreamShield ShortReuse v2). Constructed the quantitative analysis and authored Sections 10–11, including all figures, tables, and interpretations.
Dante Gencel	Computed and validated the metadata-area tables for the project’s top-performing cache-replacement policies, including derivation of per-policy storage overheads and integration of the results into the final design tradeoff analysis. Authored the sensitivity study on prompting style (complex vs. concise), including experimental setup, evaluation, and the written report summarizing its impact on stability and output quality across the CacheForge workflow.
Nick Bailey	Designed and implemented the lineage policy to track policies in a tree-like graph structure, enabling clear visualization and analysis of search trajectories. Contributed to the surrogate IPC model and its integration into the search loop, and ran multiple full and reduced-SPEC experiments to identify high-performing policies. Led the mutation-rate sensitivity study, evaluating how different mutation schedules affect IPC dynamics and search stability. Also worked on the system design for the report, helping to articulate how the end-to-end CacheForge loop operates in practice.
Om Patel	Researched and authored the introduction and conclusion section of the paper. Ran a sensitivity study between fallback policy and no fallback policy, one of the implementations that led to the creation of the best policy. Ran multiple iterations using open-source models on the CacheForge pipeline, however did not get any results. This ultimately led to us deciding to abandon the open-source path and focus fully on using paid models, specifically OpenAI models.
Rishabh Patel	Here for the vibes. Conducted background research on cache replacement policies, providing key insights to guide initial decisions. Assisted with organizing experiment logs and configuring simulation parameters. Contributed to the documentation and helped with basic troubleshooting during simulation runs.